

VIT BHOPAL UNIVERSITY
DATABASE MANAGEMENT SYSTEM
Course Code: CSE3001

PROJECT REPORT

on

LIBRARY MANAGEMENT SYSTEM

A relational database project using Java, MySQL and JDBC

Submitted by

Name	Registration No.
25BCE11104	SHIVAM RATHORE
25BCE11174	ANKUR MISHRA
25BCE11212	ADITYA SINGH
24BAI10971	ISHANGI CHOUBEY

Submitted To:

Dr Vijendra

Academic Year: 2026-27

Repository: <https://github.com/Ankur122005/Library-Management-System>

Abstract

Libraries handle a large and constantly changing volume of data: books, authors, publishers, branches, members, staff, loans, fines and reservations. Managing this information on paper or in spreadsheets leads to duplicated records, slow searches and errors in tracking who has borrowed which book. This project presents a Library Management System (LMS) built as a DBMS academic project. It uses MySQL as the relational database, Java as the application language, and JDBC as the connectivity layer.

The database, named lms, consists of fourteen related tables designed with primary keys, foreign keys, a composite key, unique and enumerated constraints, and normalized up to the third normal form. A console-based Java application exposes operations for managing books, members, authors, categories, branches and copies, issuing and returning books, loan reports, fines, reservations, reviews and aggregate reports. Book issue and return are executed as transactions so that the loan record and the available-copy count are always updated together.

Keywords: Library Management System, DBMS, MySQL, JDBC, ER Model, Normalization, SQL, Transactions

Table of Contents

Abstract	1
1. Introduction	3
1.1 Background	3
1.2 Problem Statement	3
1.3 Objectives	3
1.4 Scope	3
2. Technologies and System Requirements	4
2.1 Technologies Used	4
2.2 Software Requirements	4
2.3 Functional Requirements	4
2.4 Non-Functional Requirements	4
3. System Design	6
3.1 System Architecture	6
3.2 Entity–Relationship Diagram	7
3.3 Relationships and Cardinality	7
4. Database Design	9
4.1 Overview	9
4.2 Table Definitions	9
4.3 Keys and Constraints	13
5. Normalization	15
5.1 First Normal Form (1NF)	15
5.2 Second Normal Form (2NF)	15
5.3 Third Normal Form (3NF)	15
5.4 Example: Decomposition of a Single Library Table	15
6. Implementation	16
6.1 Project Structure	16
6.2 Application Menu	16
6.3 Database Connectivity (JDBC)	16
6.4 Issue Book Module	17
6.5 Return Book Module	17
6.6 Representative SQL Queries	17
6.7 DBMS Concepts Demonstrated	19
7. Testing	20
8. Conclusion	21
8.1 Limitations	21
8.2 Future Scope	21
Appendix A: How to Run the Project	22
References	23

1. Introduction

1.1 Background

A library is an information system in its own right. Every day it must answer questions such as: Which books are available? Which branch holds a copy? Who has borrowed a given book, and when is it due? Has a membership expired? Which books are overdue and what fine is owed? A database management system provides a structured, consistent and shareable way to store the data needed to answer these questions and to enforce the rules that keep that data valid.

1.2 Problem Statement

Manual or file-based library record keeping suffers from data redundancy, inconsistency between records, difficulty in searching, no enforcement of business rules (for example, issuing a book that has no available copy), and no reliable audit trail of circulation. The problem addressed by this project is to design and implement a relational database and a supporting application that store library data in an organized form and support the day-to-day operations of a library correctly and efficiently.

1.3 Objectives

- Design a relational database for a library with proper entities, attributes and relationships.
- Apply keys and constraints to maintain entity and referential integrity.
- Apply normalization to reduce data redundancy and avoid anomalies.
- Implement library operations (search, add, update, delete, issue, return, reports) using SQL.
- Connect a Java application to MySQL using JDBC and execute issue/return as atomic transactions.
- Demonstrate practical DBMS concepts such as joins, aggregation, grouping and transaction control through a working application.

1.4 Scope

The system covers the management of books and their authors, categories and publishers; multiple library branches and the physical copies held at each; members and their membership status; staff and their designations; circulation (issue, return, overdue detection); fines; reservations; and member reviews with average ratings. The user interface is a number-based console menu. A graphical interface, automated notifications and online payment are outside the present scope (see Future Scope).

2. Technologies and System Requirements

2.1 Technologies Used

Technology	Purpose
Java	Application development
MySQL	Relational database management (database: lms)
JDBC	Java–MySQL connectivity
Maven	Project and dependency management (pom.xml)
IntelliJ IDEA	Development environment
Git & GitHub	Version control and collaboration

2.2 Software Requirements

- Java Development Kit (JDK)
- MySQL Server
- Apache Maven
- IntelliJ IDEA (or any Java IDE)
- MySQL JDBC connector (managed through Maven)

2.3 Functional Requirements

- Book management: view, search by title or ISBN, view available books, add, update price and delete.
- Member management: view, find by card number, view active members, add, update phone number and search by name.
- Author and category viewing, including books with their authors and the number of books in each category.
- Branch and copy management: view copies by branch and find a book across branches.
- Issue a book only to an existing, active member, from an available copy, without duplicate issue of the same book to the same member.
- Return a book and restore the available copy count.
- Loan reports: currently issued books, member borrowing history and overdue books.
- Fine management: view unpaid fines, add a fine and mark a fine as paid.
- Reservations: view pending reservations, add and cancel a reservation.
- Reviews: view reviews for a book, add a review and compute average ratings.
- Reports: books by category, most borrowed books, total books, total available copies and members with active loans.

2.4 Non-Functional Requirements

- **Data integrity:** primary key, foreign key and unique constraints prevent invalid or orphaned data.
- **Consistency:** issue and return operations are transactional.
- **Maintainability:** normalized tables mean each fact is stored once.
- **Usability:** a simple numbered console menu with sub-menus for each module.
- **Security:** staff passwords are stored as hashes, and staff designations carry an access level.

3. System Design

3.1 System Architecture

The system follows a simple layered architecture. The user interacts with a console menu implemented in Main.java. Requests are translated into SQL statements and sent to MySQL through JDBC; the connection is created and managed by DBConnection.java. The database lms stores all persistent data.

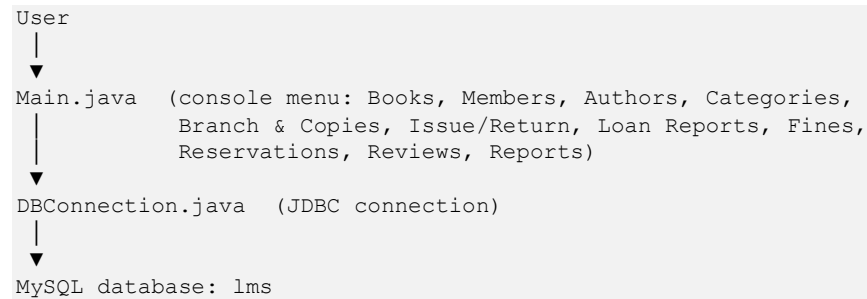


Figure 3.1: Layered architecture of the Library Management System

3.2 Entity–Relationship Diagram

The ER diagram below shows the fourteen entities of the system and the relationships between them. For readability each entity lists only its key attributes and a few representative attributes; the complete attribute list is given in Chapter 4. Crow's foot notation is used: a bar marks the "one" side and a crow's foot marks the "many" side of a relationship.

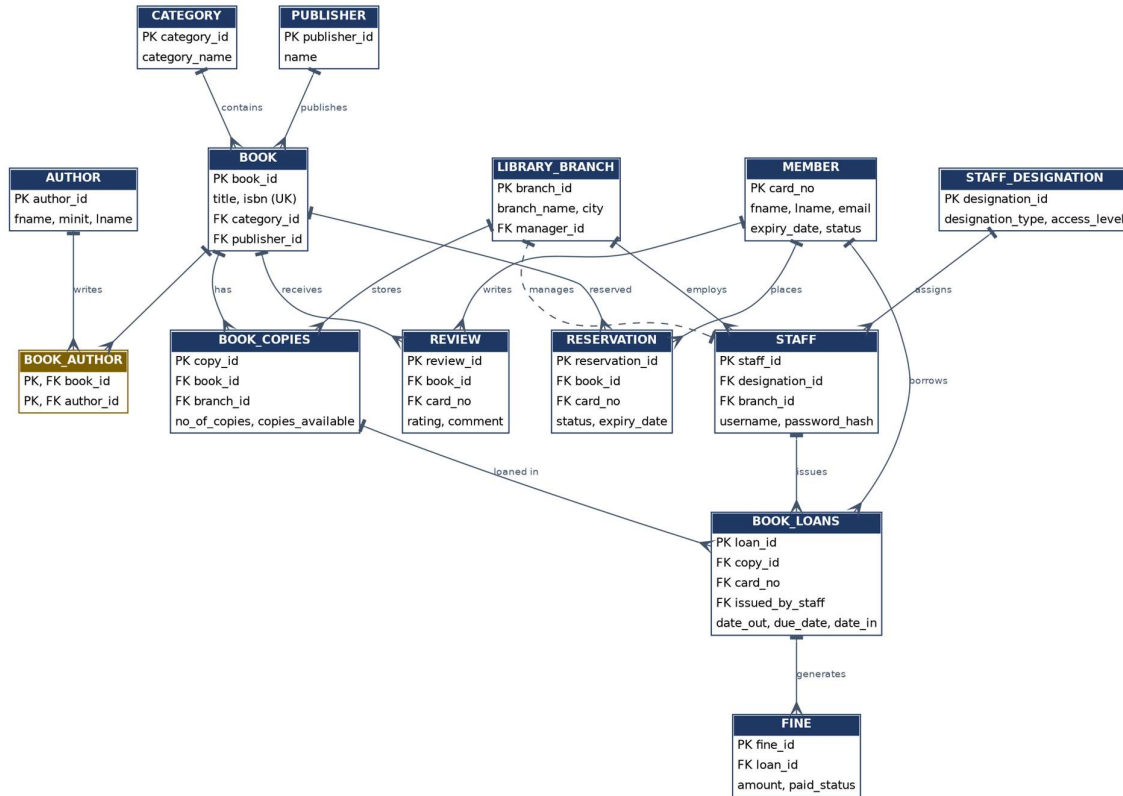


Figure 3.2: ER diagram of the Library Management System

3.3 Relationships and Cardinality

Relationship	Cardinality	Implemented by
CATEGORY – BOOK	1 : N	BOOK.category_id → CATEGORY
PUBLISHER – BOOK	1 : N	BOOK.publisher_id → PUBLISHER
BOOK – AUTHOR	M : N	Junction table BOOK_AUTHOR
BOOK – BOOK_COPIES	1 : N	BOOK_COPIES.book_id → BOOK
LIBRARY_BRANCH – BOOK_COPIES	1 : N	BOOK_COPIES.branch_id → LIBRARY_BRANCH
STAFF_DESIGNATION – STAFF	1 : N	STAFF.designation_id → STAFF_DESIGNATION
LIBRARY_BRANCH – STAFF (employs)	1 : N	STAFF.branch_id → LIBRARY_BRANCH
STAFF – LIBRARY_BRANCH (manages)	1 : 1	LIBRARY_BRANCH.manager_id → STAFF

Relationship	Cardinality	Implemented by
MEMBER – BOOK_LOANS	1 : N	BOOK_LOANS.card_no → MEMBER
STAFF – BOOK_LOANS	1 : N	BOOK_LOANS.issued_by_staff → STAFF
BOOK_COPIES – BOOK_LOANS	1 : N	BOOK_LOANS.copy_id → BOOK_COPIES
BOOK_LOANS – FINE	1 : N	FINE.loan_id → BOOK_LOANS
BOOK – RESERVATION	1 : N	RESERVATION.book_id → BOOK
MEMBER – RESERVATION	1 : N	RESERVATION.card_no → MEMBER
BOOK – REVIEW	1 : N	REVIEW.book_id → BOOK
MEMBER – REVIEW	1 : N	REVIEW.card_no → MEMBER

Note that STAFF and LIBRARY_BRANCH reference each other: a staff member works at a branch, and a branch has a staff member as its manager. This circular reference is typically handled by inserting the branch first with a null manager (or adding the manager foreign key after both tables exist) and then updating it.

4. Database Design

4.1 Overview

The database lms is organized into fourteen tables instead of a single table holding all library information. This separation reduces redundancy, makes updates safe, and allows each kind of fact to be stored exactly once. The tables are summarized below.

Table	Purpose
BOOK	Stores book information
AUTHOR	Stores author information
BOOK_AUTHOR	Junction table for the many-to-many relationship between books and authors
CATEGORY	Stores book categories
PUBLISHER	Stores publisher information
LIBRARY_BRANCH	Stores library branch information
BOOK_COPIES	Stores physical copy information for each book at each branch
MEMBER	Stores library member information
STAFF	Stores library staff information
STAFF_DESIGNATION	Stores staff designation and access level
BOOK_LOANS	Stores book issue and return (circulation) records
FINE	Stores fines generated by loans
RESERVATION	Stores book reservations
REVIEW	Stores member reviews of books

4.2 Table Definitions

4.2.1 BOOK

Stores book information.

Column	Data Type	Key / Description
book_id	INT	Primary Key
title	VARCHAR(200)	Book title
isbn	VARCHAR(20)	Unique ISBN
category_id	INT	FK → CATEGORY
publisher_id	INT	FK → PUBLISHER
language	VARCHAR(30)	Book language
edition	VARCHAR(20)	Book edition
publication_year	YEAR	Publication year
price	DECIMAL(8,2)	Book price

4.2.2 AUTHOR

Stores author information.

Column	Data Type	Key / Description
author_id	INT	Primary Key
fname	VARCHAR(50)	First name
minit	CHAR(1)	Middle initial, nullable
lname	VARCHAR(50)	Last name

4.2.3 BOOK_AUTHOR

Junction table for the many-to-many relationship between books and authors.

Column	Data Type	Key / Description
book_id	INT	PK, FK → BOOK
author_id	INT	PK, FK → AUTHOR

4.2.4 CATEGORY

Stores book categories.

Column	Data Type	Key / Description
category_id	INT	Primary Key
category_name	VARCHAR(60)	Category name
description	VARCHAR(255)	Optional description

4.2.5 PUBLISHER

Stores publisher information.

Column	Data Type	Key / Description
publisher_id	INT	Primary Key
name	VARCHAR(100)	Publisher name
address	VARCHAR(200)	Publisher address
contact_no	VARCHAR(15)	Contact number

4.2.6 LIBRARY_BRANCH

Stores library branch information.

Column	Data Type	Key / Description
branch_id	INT	Primary Key

Column	Data Type	Key / Description
branch_name	VARCHAR(80)	Branch name
address	VARCHAR(200)	Address
city	VARCHAR(50)	City
state	VARCHAR(50)	State
zip_code	VARCHAR(10)	Postal code
contact_no	VARCHAR(15)	Contact number
manager_id	INT	FK → STAFF

4.2.7 BOOK_COPIES

Stores physical copy information for each book at each branch.

Column	Data Type	Key / Description
copy_id	INT	Primary Key
book_id	INT	FK → BOOK
branch_id	INT	FK → LIBRARY_BRANCH
no_of_copies	INT	Total copies
copies_available	INT	Available copies
rental_category	ENUM	Rentable / Not_Rentable
rental_price	DECIMAL(6,2)	Rental price, nullable

4.2.8 MEMBER

Stores library member information.

Column	Data Type	Key / Description
card_no	INT	Primary Key
fname	VARCHAR(50)	First name
lname	VARCHAR(50)	Last name
address	VARCHAR(200)	Address
phone	VARCHAR(15)	Phone number
email	VARCHAR(100)	Email
campus_id	VARCHAR(20)	Optional campus ID
registration_date	DATE	Registration date
expiry_date	DATE	Membership expiry
status	ENUM	Active / Expired / Suspended

4.2.9 STAFF

Stores library staff information.

Column	Data Type	Key / Description
staff_id	INT	Primary Key
name	VARCHAR(80)	Staff name
designation_id	INT	FK → STAFF_DESIGNATION
branch_id	INT	FK → LIBRARY_BRANCH
contact_no	VARCHAR(15)	Contact number
username	VARCHAR(30)	System username
password_hash	VARCHAR(255)	Password hash

4.2.10 STAFF_DESIGNATION

Stores staff designation and access level.

Column	Data Type	Key / Description
designation_id	INT	Primary Key
designation_type	VARCHAR(40)	Designation
access_level	TINYINT	Access level

4.2.11 BOOK_LOANS

Stores book issue and return (circulation) records.

Column	Data Type	Key / Description
loan_id	INT	Primary Key
copy_id	INT	FK → BOOK_COPIES
card_no	INT	FK → MEMBER
issued_by_staff	INT	FK → STAFF
date_out	DATE	Issue date
due_date	DATE	Due date
date_in	DATE	Return date, nullable
lend_type	VARCHAR(20)	Lending type
grace_period	INT	Grace period in days, nullable

4.2.12 FINE

Stores fines generated by loans.

Column	Data Type	Key / Description
fine_id	INT	Primary Key

Column	Data Type	Key / Description
loan_id	INT	FK → BOOK_LOANS
amount	DECIMAL(6,2)	Fine amount
reason	VARCHAR(100)	Reason for fine
paid_status	ENUM	Paid / Unpaid
paid_date	DATE	Payment date, nullable

4.2.13 RESERVATION

Stores book reservations.

Column	Data Type	Key / Description
reservation_id	INT	Primary Key
book_id	INT	FK → BOOK
card_no	INT	FK → MEMBER
reservation_date	DATE	Reservation date
status	ENUM	Pending / Fulfilled / Cancelled
expiry_date	DATE	Reservation expiry

4.2.14 REVIEW

Stores member reviews of books.

Column	Data Type	Key / Description
review_id	INT	Primary Key
book_id	INT	FK → BOOK
card_no	INT	FK → MEMBER
rating	TINYINT	Rating from 1 to 5
comment	VARCHAR(255)	Optional comment
review_date	DATE	Review date

4.3 Keys and Constraints

Constraint	Where used	Purpose
PRIMARY KEY	Every table (e.g., BOOK.book_id, MEMBER.card_no, BOOK_LOANS.loan_id)	Uniquely identifies each row
Composite PRIMARY KEY	BOOK_AUTHOR (book_id, author_id)	Identifies a book–author pair; resolves the M:N relationship
FOREIGN KEY	BOOK.category_id, BOOK_LOANS.card_no, FINE.loan_id, etc.	Maintains referential integrity between tables

Constraint	Where used	Purpose
UNIQUE	BOOK.isbn	No two books may share an ISBN
ENUM	MEMBER.status, FINE.paid_status, RESERVATION.status, BOOK_COPIES.rental_category	Restricts a column to a fixed set of valid values
NULL / NOT NULL	e.g., AUTHOR.minit, BOOK_LOANS.date_in	date_in is NULL while a book is still issued
Rating scale	REVIEW.rating (TINYINT, 1–5)	Ratings are on a 1 to 5 scale

5. Normalization

Normalization is the process of organizing data to reduce redundancy and avoid insertion, update and deletion anomalies. The LMS schema is designed to satisfy the first three normal forms.

5.1 First Normal Form (1NF)

Every column holds a single atomic value and there are no repeating groups.

A book with several authors is not stored as a list of names inside BOOK; instead each book–author pair is a separate row of BOOK_AUTHOR. Author names are split into fname, minit and lname.

5.2 Second Normal Form (2NF)

There are no partial dependencies on part of a composite key.

The only composite key is in BOOK_AUTHOR, whose two columns together form the key and which has no other attributes.

All other tables use a single-column primary key, so every non-key attribute depends on the whole key.

5.3 Third Normal Form (3NF)

There are no transitive dependencies among non-key attributes.

Category details are stored in CATEGORY and BOOK stores only category_id; publisher details are in PUBLISHER; staff designation and access level are in STAFF_DESIGNATION rather than being repeated on every STAFF row; and fines are stored in FINE rather than inside BOOK_LOANS.

5.4 Example: Decomposition of a Single Library Table

Suppose all data were kept in one table: LIBRARY(book_title, isbn, category_name, publisher_name, publisher_address, author_names, branch_name, member_name, date_out, due_date, fine_amount). The publisher address would be repeated for every book of that publisher (update anomaly), a new category could not be recorded without a book (insertion anomaly), and deleting the last loan of a member would erase the member's details (deletion anomaly). Decomposing this table into BOOK, CATEGORY, PUBLISHER, AUTHOR, BOOK_AUTHOR, LIBRARY_BRANCH, MEMBER, BOOK_LOANS and FINE removes these anomalies.

6. Implementation

6.1 Project Structure

```
Library-Management-System/
├── src/main/java/com/hms/
│   ├── Main.java           (console application and menus)
│   └── DBConnection.java    (JDBC connection to MySQL)
├── sql/
│   └── library_queries.sql  (table creation and queries)
├── pom.xml                 (Maven configuration)
├── run_library.bat          (Windows launcher)
├── .gitignore
└── README.md
```

Figure 6.1: Project directory structure

6.2 Application Menu

The application uses a number-based console interface. The main menu offers the following options, most of which open a sub-menu.

No.	Module	Sub-operations
1	Books	View; search by title; search by ISBN; view available; add; update price; delete
2	Members	View; find by card no.; view active; add; update phone; search by name
3	Authors	View authors; view books with authors
4	Categories	View categories; count books by category
5	Branch & Copies	View copies by branch; find a book across branches
6	Issue Book	Issue an available copy to an active member
7	Return Book	Return an issued book
8	Loan Reports	Currently issued; member borrowing history; overdue books
9	Fines	View unpaid; add fine; mark as paid
10	Reservations	View pending; add; cancel
11	Reviews	View reviews for a book; add review; average rating per book
12	Reports	Books by category; most borrowed; total books; total available copies; members with active loans
13	Exit	Close the application

6.3 Database Connectivity (JDBC)

The Java application communicates with MySQL through JDBC. The connection URL, user name and password are configured in DBConnection.java, and Main.java obtains a connection from this class for every database operation. Parameterized statements (PreparedStatement) are the recommended way to pass user input into SQL to avoid SQL injection.

6.4 Issue Book Module

Issuing a book involves several checks and two table updates, so it is executed as one transaction. The application:

1. Checks that the member exists and is active.
2. Prevents the same member from having the same book issued twice.
3. Finds an available physical copy.
4. Selects an existing staff member as the issuing staff.
5. Creates the loan record in BOOK_LOANS.
6. Decreases copies_available in BOOK_COPIES.
7. Commits both changes together.

```
START TRANSACTION;
INSERT INTO BOOK_LOANS (copy_id, card_no, issued_by_staff, date_out, due_date,
lend_type)
VALUES (?, ?, ?, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 14 DAY), ?);

UPDATE BOOK_COPIES
SET copies_available = copies_available - 1
WHERE copy_id = ? AND copies_available > 0;
COMMIT;
```

Figure 6.2: Issue book transaction (illustrative; the loan period shown is an example)

6.5 Return Book Module

To return a book, the application finds the active loan (a row with date_in IS NULL) for the selected member and book, sets date_in, and increases the available copy count. Both updates are committed as one transaction, so the loan status and the copy count can never disagree.

```
START TRANSACTION;
UPDATE BOOK_LOANS
SET date_in = CURDATE()
WHERE loan_id = ? AND date_in IS NULL;

UPDATE BOOK_COPIES
SET copies_available = copies_available + 1
WHERE copy_id = ?;
COMMIT;
```

Figure 6.3: Return book transaction (illustrative)

6.6 Representative SQL Queries

The following queries illustrate the SQL features used by the project: joins, filtering, aggregation, grouping and date functions. They are written against the schema in Chapter 4.

(a) View books with category and publisher (JOIN)

```
SELECT b.book_id, b.title, b.isbn, c.category_name, p.name AS publisher, b.price
FROM BOOK b
JOIN CATEGORY c ON b.category_id = c.category_id
JOIN PUBLISHER p ON b.publisher_id = p.publisher_id;
```

(b) Search a book by title

```
SELECT book_id, title, isbn, price
FROM BOOK
WHERE title LIKE CONCAT('%', ?, '%');
```

(c) Books with their authors (many-to-many JOIN)

```
SELECT b.title, a.fname, a.lname
FROM BOOK b
JOIN BOOK_AUTHOR ba ON b.book_id = ba.book_id
JOIN AUTHOR a ON ba.author_id = a.author_id
ORDER BY b.title;
```

(d) Count of books in each category (COUNT with GROUP BY)

```
SELECT c.category_name, COUNT(b.book_id) AS total_books
FROM CATEGORY c
LEFT JOIN BOOK b ON b.category_id = c.category_id
GROUP BY c.category_name;
```

(e) Overdue books (date functions)

```
SELECT m.card_no, m.fname, b.title, l.due_date,
       DATEDIFF(CURDATE(), l.due_date) AS days_overdue
FROM BOOK_LOANS l
JOIN MEMBER m ON l.card_no = m.card_no
JOIN BOOK_COPIES bc ON l.copy_id = bc.copy_id
JOIN BOOK b ON bc.book_id = b.book_id
WHERE l.date_in IS NULL
AND l.due_date < CURDATE();
```

(f) Average rating of each book (AVG, COUNT, ROUND)

```
SELECT b.title, ROUND(AVG(r.rating), 2) AS avg_rating, COUNT(r.review_id) AS
reviews
FROM BOOK b
JOIN REVIEW r ON b.book_id = r.book_id
GROUP BY b.book_id, b.title;
```

(g) Most borrowed books (aggregation with ORDER BY)

```
SELECT b.title, COUNT(l.loan_id) AS times_borrowed
FROM BOOK_LOANS l
JOIN BOOK_COPIES bc ON l.copy_id = bc.copy_id
JOIN BOOK b ON bc.book_id = b.book_id
GROUP BY b.book_id, b.title
ORDER BY times_borrowed DESC;
```

(h) Unpaid fines

```
SELECT f.fine_id, m.fname, m.lname, f.amount, f.reason
FROM FINE f
JOIN BOOK_LOANS l ON f.loan_id = l.loan_id
JOIN MEMBER m ON l.card_no = m.card_no
WHERE f.paid_status = 'Unpaid';
```

(i) Total available copies (SUM)

```
| SELECT SUM(copies_available) AS total_available_copies FROM BOOK_COPIES;
```

6.7 DBMS Concepts Demonstrated

Concept	Where it appears in the project
Relational model	Fourteen related tables in the lms database
Primary / foreign / composite keys	All tables; BOOK_AUTHOR uses a composite key
Cardinality (1:N, M:N)	Category–Book, Member–Loans, Book–Author via junction table
Normalization (1NF–3NF)	Separate tables for category, publisher, designation, fine, etc.
Constraints	PRIMARY KEY, FOREIGN KEY, UNIQUE (ISBN), ENUM status columns
Joins	Book listing, books with authors, overdue books, copies by branch
Aggregate functions & GROUP BY	COUNT, SUM, AVG, ROUND in category counts, reports and ratings
Date functions	CURDATE() and DATEDIFF() for overdue detection
Transactions (ACID)	Issue and return commit loan and copy updates atomically
JDBC	Java application connected to MySQL via DBConnection.java

7. Testing

The following test scenarios verify the main business rules of the system. The "Actual result" column is to be filled in by the team after running each scenario on the application.

No.	Scenario	Expected result
1	Search a book by an existing title	Matching book(s) displayed with details
2	Search a book by an ISBN that does not exist	Message that no book was found
3	Add a book with an ISBN that already exists	Insert rejected by the UNIQUE constraint
4	Issue a book to an active member	Loan row created; copies_available reduced by 1
5	Issue a book to an expired or suspended member	Issue refused
6	Issue the same book again to the same member	Issue refused (duplicate)
7	Issue a book with no available copy	Issue refused
8	Return an issued book	date_in set; copies_available increased by 1
9	View overdue books after a due date has passed	Book listed with number of overdue days
10	Delete a category that still has books	Rejected by the foreign key constraint
11	Add a review with a rating outside 1–5	Rejected (by a database check or application validation)
12	View average rating per book	Average and review count shown for each reviewed book

8. Conclusion

The Library Management System demonstrates how a well-designed relational database supports the real operations of a library. The fourteen-table schema, designed with appropriate keys and constraints and normalized to 3NF, stores each fact once and keeps related data consistent. The Java–JDBC application makes the database usable through a simple console interface and shows important DBMS ideas in practice: joins, aggregate queries, constraints and atomic transactions for issuing and returning books. The project met its objectives of designing the database, applying normalization, implementing library operations in SQL and connecting an application to MySQL.

8.1 Limitations

- The interface is console-based and has no graphical user interface.
- Fines are added through a menu operation rather than being calculated automatically at return time.
- There are no automatic notifications for due dates or reservation availability.
- The database connection settings are edited directly in the source code.

8.2 Future Scope

- A graphical or web-based user interface.
- Automatic fine calculation based on overdue days and the grace period.
- Reservation and due-date notifications by e-mail or SMS.
- Advanced search (by author, category, language and year).
- Role-based screens for members, librarians and administrators, using the staff access level.
- More reports and statistics, and support for additional library branches.

Appendix A: How to Run the Project

Prerequisites

- Java JDK
- MySQL
- IntelliJ IDEA
- Maven

Steps

1. Create the database:

```
CREATE DATABASE lms;  
USE lms;
```

2. Create the tables by running sql/library_queries.sql.
3. Update the MySQL user name and password in src/main/java/com/hms/DBConnection.java.
4. Run the Main class from IntelliJ IDEA, or on Windows start the application with run_library.bat.

References

1. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, Pearson.
2. A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, McGraw-Hill.
3. MySQL 8.0 Reference Manual, Oracle Corporation. <https://dev.mysql.com/doc/>
4. JDBC Basics, Oracle Java Tutorials. <https://docs.oracle.com/javase/tutorial/jdbc/>
5. Project repository: <https://github.com/Ankur122005/Library-Management-System>